🚀 The ULTIMATE Whisper Small Fine-Tuning Pipeline

This version is designed specifically to handle **7-10+ hours** of dataset generation without crashing Google Colab.

Key Upgrades in this version:

- **Automatic Disk Cleanup:** Audio files (`.wav`) are massive. This script deletes them immediately after their spectrogram features are generated to prevent Colab from running out of disk space!
- **Google Drive Integration (Optional):** Automatically saves the finished fine-tuned model to your Google Drive to prevent data loss if the session disconnects.
- **Bulletproof Transcripts:** Automatically parses all languages, converts `nl`, `es`, `de`, `fr`, etc. native subtitles to `en`, and safely formats them into JSON dictionaries.
- **Batch Safety:** If a corrupted chunk fails during processing, it intelligently logs it and continues processing the rest of the 500+ chunks instead of crashing the notebook.

Important: Change runtime type > Hardware accelerator > **T4 GPU**.

Step 0: Mount Google Drive (Recommended but Optional)

```
# @title Step 0: Mount Google Drive (Recommended but Optional)
# This ensures your 1-2 hour trained model is safely stored on your Drive instantly.

from google.colab import drive
try:
    drive.mount('/content/drive')
    print("\n✅ Google Drive mounted correctly! Models will be saved here.")
    DRIVE_AVAILABLE = True
except Exception as e:
    print("\n⚠️ Could not mount Drive. Models will be saved to local Colab storage only.")
    DRIVE_AVAILABLE = False
```

```
Mounted at /content/drive

✅ Google Drive mounted correctly! Models will be saved here.
```

Step 1: Install Dependencies

```
!pip install torch torchvision torchaudio
!pip install transformers datasets accelerate evaluate jiwer librosa soundfile yt-dlp youtube-transcript-api tqd
!apt-get install -y ffmpeg
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.24.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (1
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torc
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindi
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvisi
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.12.0)
```

```
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Collecting jiwer
  Downloading jiwer-4.0.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: librosa in /usr/local/lib/python3.12/dist-packages (0.11.0)
Requirement already satisfied: soundfile in /usr/local/lib/python3.12/dist-packages (0.13.1)
Collecting yt-dlp
  Downloading yt_dlp-2026.3.3-py3-none-any.whl.metadata (182 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 182.2/182.2 kB 16.9 MB/s eta 0:00:00
Collecting youtube-transcript-api
  Downloading youtube_transcript_api-1.2.4-py3-none-any.whl.metadata (24 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (0.25.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.3)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from trai
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (2
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from trai
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers
```

Step 2: Download Scripts

We combine everything into two highly optimized scripts.

```
%%writefile 1_download_and_extract.py
import os
import json
import logging
from typing import List, Dict, Optional
import yt_dlp
from youtube_transcript_api import YouTubeTranscriptApi

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

DATA_DIR = os.path.join(os.getcwd(), 'dataset', 'raw')
AUDIO_DIR = os.path.join(DATA_DIR, 'audio')
TRANSCRIPT_DIR = os.path.join(DATA_DIR, 'transcripts')
os.makedirs(AUDIO_DIR, exist_ok=True)
os.makedirs(TRANSCRIPT_DIR, exist_ok=True)

def extract_video_id(url: str) -> str:
    if "youtu.be" in url: return url.split("/")[-1].split("?")[0]
    if "v=" in url: return url.split("v=")[1].split("&")[0]
    return url

def get_urls_from_input(url: str) -> List[str]:
    ydl_opts = {'extract_flat': 'in_playlist', 'quiet': True, 'no_warnings': True}
    try:
        with yt_dlp.YoutubeDL(ydl_opts) as ydl:
            info = ydl.extract_info(url, download=False)
            if 'entries' in info:
                logger.info(f"Playlist detected! Found {len(info['entries'])} videos.")
                return [entry['url'] for entry in info['entries'] if entry.get('url')]
            return [url]
    except Exception as e:
        logger.error(f"Failed to extract info from {url}. Ensure it is a valid link (e.g. ?list=ID). Error: {e}"
        return []

def download_youtube_audio(youtube_url: str, output_path: str) -> bool:
    logger.info(f"Downloading audio from {youtube_url}...")
    ydl_opts = {
        'format': 'bestaudio/best',
        'postprocessors': [{'key': 'FFmpegExtractAudio', 'preferredcodec': 'wav'}],
        'postprocessor_args': ['-ar', '16000', '-ac', '1'], # 16kHz mono required for Whisper
        'outtmpl': output_path, # Fixed output template (.wav gets added by yt-dlp automatically)
        'quiet': False,
        'no_warnings': True,
    }
    try:
        with yt_dlp.YoutubeDL(ydl_opts) as ydl: ydl.download([youtube_url])
        return True
    except Exception as e:
        logger.error(f"Failed to download audio for {youtube_url}: {e}")
        return False

def download_youtube_transcript(video_id: str, output_path: str, languages: List[str] = ['en', 'nl', 'es', 'de',
    logger.info(f"Fetching transcript for video ID: {video_id}...")
```

```
    try:
        # Fixed: Usage of instance method based on modern youtube_transcript_api versions!!!
        transcript_list = YouTubeTranscriptApi().list(video_id)

        # Try to find transcript in our widespread supported language list
        transcript = transcript_list.find_transcript(languages)

        # Always normalize and translate to English to guarantee Whisper learns English text mapping
        if transcript.language_code != 'en':
            logger.info(f"Translating video {video_id} ({transcript.language_code}) native transcript to English
            transcript = transcript.translate('en')

        fetched_data = transcript.fetch()

        # Fixed: Safely enforce JSON Serialization from FetchedTranscriptSnippet using attribute dot-notation
        data = [
            {"text": t.text, "start": t.start, "duration": t.duration}
            for t in fetched_data
        ]

        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        return data

    except Exception as e:
        logger.error(f"Failed to fetch transcript for {video_id}: {e}")
        return None

def process_video(url: str):
    video_id = extract_video_id(url)
    logger.info(f"\n{'='*40}\nProcessing Video ID: {video_id}\n{'='*40}")

    transcript_data = download_youtube_transcript(video_id, os.path.join(TRANSCRIPT_DIR, f"{video_id}.json"))
    if not transcript_data:
        logger.warning(f"Skipping {video_id} due to missing transcripts.")
        return False

    audio_path = os.path.join(AUDIO_DIR, video_id)
    if not os.path.exists(f"{audio_path}.wav"):
        if not download_youtube_audio(url, audio_path): return False
    return True

if __name__ == "__main__":
    import sys
    input_urls = [u.strip() for u in sys.argv[1].split(",") if u.strip()]
    target_urls = []
    for u in input_urls: target_urls.extend(get_urls_from_input(u))
    logger.info(f"Total discovered videos to process: {len(target_urls)}")
    for url in target_urls: process_video(url)
    logger.info("\n\n✅ Phase 1: Downloading & Extraction Completed! ✓")
```

```
Writing 1_download_and_extract.py
```

```
%%writefile 2_build_dataset.py
import os
import json
import logging
from pydub import AudioSegment
from datasets import Dataset, DatasetDict
from transformers import WhisperFeatureExtractor, WhisperTokenizer
import warnings
import shutil

warnings.filterwarnings("ignore")
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')
logger = logging.getLogger(__name__)

BASE_DIR = os.getcwd()
DATA_DIR = os.path.join(BASE_DIR, 'dataset', 'raw')
PROCESSED_DATA_DIR = os.path.join(BASE_DIR, 'dataset', 'processed')
os.makedirs(PROCESSED_DATA_DIR, exist_ok=True)

MODEL_ID = "openai/whisper-small"
MAX_DURATION = 30000  # 30 seconds explicitly
MIN_DURATION = 500    # 500 ms minimum

def main():
    logger.info("🚀 Initializing High-Volume Preprocessing Pipeline...")
    feature_extractor = WhisperFeatureExtractor.from_pretrained(MODEL_ID)
    tokenizer = WhisperTokenizer.from_pretrained(MODEL_ID, language="english", task="transcribe")

    # Step A: Identify valid transcripts & audio
```

```
valid_pairs = []
if not os.path.exists(os.path.join(DATA_DIR, 'transcripts')):
    logger.error("Dataset raw directories do not exist! Did Step 1 finish successfully?")
    return

for f in os.listdir(os.path.join(DATA_DIR, 'transcripts')):
    if f.endswith('.json'):
        vid = f.replace('.json', '')
        audio_path = os.path.join(DATA_DIR, 'audio', f"{vid}.wav")
        if os.path.exists(audio_path):
            valid_pairs.append((vid, audio_path, os.path.join(DATA_DIR, 'transcripts', f)))

if not valid_pairs:
    logger.error("❌ 0 valid Audio + Transcript pairs detected!")
    return

logger.info(f"🔪 Slicing {len(valid_pairs)} master videos into Whisper 30s Chunks...")
dataset_rows = []

# Step B: Slice Audio
for vid, audio_path, trans_path in valid_pairs:
    with open(trans_path, 'r') as file:
        transcript_data = json.load(file)

    try:
        audio = AudioSegment.from_wav(audio_path)
    except Exception as e:
        logger.warning(f"Could not load Audio file for {vid}: {e}. Skipping.")
        continue

    chunk_dir = os.path.join(PROCESSED_DATA_DIR, 'chunks', vid)
    os.makedirs(chunk_dir, exist_ok=True)

    successful_chunks = 0
    for i, seg in enumerate(transcript_data):
        # Safely check missing array properties for malformed YouTube captions
        if 'start' not in seg or 'duration' not in seg or 'text' not in seg: continue
        if seg['start'] is None or seg['duration'] is None: continue

        start = int(seg['start'] * 1000)
        dur = int(seg['duration'] * 1000)
        text = str(seg['text']).strip()

        # Restrict duration sizing
        if not text or dur < MIN_DURATION or dur > MAX_DURATION: continue

        chunk_path = os.path.join(chunk_dir, f"{vid}_chunk_{i}.wav")
        try:
            # Explicitly conform to Whisper format and save instantly
            sliced_audio = audio[start:start+dur]
            sliced_audio.set_frame_rate(16000).set_channels(1).export(chunk_path, format="wav")
            dataset_rows.append({"audio_path": chunk_path, "text": text})
            successful_chunks += 1
        except Exception as e:
            pass # Suppress minor slicing corruptions, keep going

    logger.info(f" > Video {vid}: Generated {successful_chunks} clean training chunks.")

    # 🧹 AGGRESSIVE DISK CLEANUP: Delete the massive raw 1-2GB origin WAV file after slicing immediately to r
    os.remove(audio_path)

if not dataset_rows:
    logger.error("❌ 0 total audio chunks properly identified. Check your video URLs.")
    return

logger.info(f"\n🔥 Ready to process {len(dataset_rows)} Total Training Chunks!")
raw_dataset = Dataset.from_list(dataset_rows)

def load_audio(batch):
    import librosa
    speech_array, sr = librosa.load(batch["audio_path"], sr=16000)
    batch["audio"] = {"array": speech_array, "sampling_rate": sr}
    return batch

def prepare_dataset(b):
    b["input_features"] = feature_extractor(b["audio"]["array"], sampling_rate=16000).input_features[0]
    b["labels"] = tokenizer(b["text"]).input_ids
    return b

# Step C: LogMel Spectrogram Extraction
logger.info("Extracting audio matrices... (This is CPU bound and may take 20-30+ minutes for huge datasets)")
# Removed num_proc=2 to prevent Colab Memory Crash (BrokenPipeError) when loading 5800+ audio files into RAM.
audio_dataset = raw_dataset.map(load_audio)
```

```
    logger.info("Tokenizing words to integer matrices...")
    processed_dataset = audio_dataset.map(prepare_dataset, remove_columns=audio_dataset.column_names)

    # 🧹 AGGRESSIVE DISK CLEANUP: Delete the small chunk files entirely since features are now in memory.
    logger.info("Wiping intermediate disk files to free storage space...")
    try:
        shutil.rmtree(os.path.join(PROCESSED_DATA_DIR, 'chunks'))
    except: pass

    # Step D: Final Output
    split_dataset = processed_dataset.train_test_split(test_size=0.1)
    save_path = os.path.join(PROCESSED_DATA_DIR, "whisper_dataset")
    split_dataset.save_to_disk(save_path)

    logger.info(f"\n✅ Phase 2: Processing Complete! Dataset safely housed mathematically at: {save_path}")

if __name__ == "__main__":
    main()
```

```
Writing 2_build_dataset.py
```

Step 3: Configure Target Videos & Run Downloads

Specify your PLAYLIST link or individual videos separated by commas. Wait for it to finish downloading the MP3s and formatting the English Transcripts.

```
YOUTUBE_URLS = "https://www.youtube.com/playlist?list=PLsRNoUx8w3rPPvAvyw4Uj_XTU-1WqD8Et"

!python 1_download_and_extract.py "{YOUTUBE_URLS}"
```

```
2026-03-09 16:02:28,306 - __main__ - INFO - Playlist detected! Found 25 videos.
2026-03-09 16:02:28,308 - __main__ - INFO - Total discovered videos to process: 25
2026-03-09 16:02:28,309 - __main__ - INFO -
========================================
Processing Video ID: eVFzbxmKNUw
========================================
2026-03-09 16:02:28,309 - __main__ - INFO - Fetching transcript for video ID: eVFzbxmKNUw...
2026-03-09 16:02:29,270 - __main__ - INFO - Downloading audio from https://www.youtube.com/watch?v=eVFzbxmKNUw.
[youtube] Extracting URL: https://www.youtube.com/watch?v=eVFzbxmKNUw
[youtube] eVFzbxmKNUw: Downloading webpage
[youtube] eVFzbxmKNUw: Downloading android vr player API JSON
[info] eVFzbxmKNUw: Downloading 1 format(s): 251
[download] Destination: /content/dataset/raw/audio/eVFzbxmKNUw
[download] 100% of   11.46MiB in 00:00:00 at 47.36MiB/s
[ExtractAudio] Destination: /content/dataset/raw/audio/eVFzbxmKNUw.wav
Deleting original file /content/dataset/raw/audio/eVFzbxmKNUw (pass -k to keep)
2026-03-09 16:02:41,821 - __main__ - INFO -
========================================
Processing Video ID: f_N3PGvnVKg
========================================
2026-03-09 16:02:41,823 - __main__ - INFO - Fetching transcript for video ID: f_N3PGvnVKg...
2026-03-09 16:02:42,767 - __main__ - INFO - Downloading audio from https://www.youtube.com/watch?v=f_N3PGvnVKg.
[youtube] Extracting URL: https://www.youtube.com/watch?v=f_N3PGvnVKg
[youtube] f_N3PGvnVKg: Downloading webpage
[youtube] f_N3PGvnVKg: Downloading android vr player API JSON
[info] f_N3PGvnVKg: Downloading 1 format(s): 251
[download] Destination: /content/dataset/raw/audio/f_N3PGvnVKg
[download] 100% of    7.92MiB in 00:00:00 at 40.52MiB/s
[ExtractAudio] Destination: /content/dataset/raw/audio/f_N3PGvnVKg.wav
Deleting original file /content/dataset/raw/audio/f_N3PGvnVKg (pass -k to keep)
2026-03-09 16:02:49,194 - __main__ - INFO -
========================================
Processing Video ID: 2XZ9z6OewR0
========================================
2026-03-09 16:02:49,198 - __main__ - INFO - Fetching transcript for video ID: 2XZ9z6OewR0...
2026-03-09 16:02:50,212 - __main__ - INFO - Downloading audio from https://www.youtube.com/watch?v=2XZ9z6OewR0.
[youtube] Extracting URL: https://www.youtube.com/watch?v=2XZ9z6OewR0
[youtube] 2XZ9z6OewR0: Downloading webpage
[youtube] 2XZ9z6OewR0: Downloading android vr player API JSON
[info] 2XZ9z6OewR0: Downloading 1 format(s): 251
[download] Destination: /content/dataset/raw/audio/2XZ9z6OewR0
[download] 100% of   11.17MiB in 00:00:00 at 42.24MiB/s
[ExtractAudio] Destination: /content/dataset/raw/audio/2XZ9z6OewR0.wav
Deleting original file /content/dataset/raw/audio/2XZ9z6OewR0 (pass -k to keep)
2026-03-09 16:03:01,899 - __main__ - INFO -
========================================
Processing Video ID: WWS086AY1Os
========================================
2026-03-09 16:03:01,899 - __main__ - INFO - Fetching transcript for video ID: WWS086AY1Os...
2026-03-09 16:03:02,762 - __main__ - INFO - Translating video WWS086AY1Os (nl) native transcript to English...
2026-03-09 16:03:03,092 - __main__ - ERROR - Failed to fetch transcript for WWS086AY1Os:
Could not retrieve a transcript for the video https://www.youtube.com/watch?v=WWS086AY1Os! This is most likely
```

```
YouTube is blocking requests from your IP. This usually is due to one of the following reasons:
- You have done too many requests and your IP has been blocked by YouTube
- You are doing requests from an IP belonging to a cloud provider (like AWS, Google Cloud Platform, Azure, etc.
```

Step 4: Build The Hugging Face Dataset

Converts audio into math! Slices everything up and maps tokens. (Will take several minutes)

```
!python 2_build_dataset.py
```

```
2026-03-09 16:05:18,764 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:18,773 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
tokenizer_config.json: 283kB [00:00, 112MB/s]
2026-03-09 16:05:19,076 - HTTP Request: GET https://huggingface.co/api/models/openai/whisper-small/tree/main/ad
2026-03-09 16:05:19,304 - HTTP Request: GET https://huggingface.co/api/models/openai/whisper-small/tree/main?re
2026-03-09 16:05:19,555 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/vocab.jso
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate li
2026-03-09 16:05:19,558 - Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKE
2026-03-09 16:05:19,564 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:19,571 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
vocab.json: 836kB [00:00, 78.5MB/s]
2026-03-09 16:05:19,818 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/tokenizer
2026-03-09 16:05:19,826 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:19,837 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
tokenizer.json: 2.48MB [00:00, 64.2MB/s]
2026-03-09 16:05:20,111 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/merges.tx
2026-03-09 16:05:20,124 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:20,137 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
merges.txt: 494kB [00:00, 55.7MB/s]
2026-03-09 16:05:20,389 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/normalize
2026-03-09 16:05:20,396 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:20,409 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
normalizer.json: 52.7kB [00:00, 13.4MB/s]
2026-03-09 16:05:20,649 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/added_tok
2026-03-09 16:05:20,664 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:20,687 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
added_tokens.json: 34.6kB [00:00, 55.1MB/s]
2026-03-09 16:05:20,921 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/special_t
2026-03-09 16:05:20,933 - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-sma
2026-03-09 16:05:20,949 - HTTP Request: GET https://huggingface.co/api/resolve-cache/models/openai/whisper-smal
special_tokens_map.json: 2.19kB [00:00, 6.44MB/s]
2026-03-09 16:05:21,186 - HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/chat_temp
2026-03-09 16:05:22,316 - 🔪 Slicing 14 master videos into Whisper 30s Chunks...
2026-03-09 16:05:22,463 -  > Video m8WomdCLBqE: Generated 263 clean training chunks.
2026-03-09 16:05:22,710 -  > Video 3M8ZxQiK4qo: Generated 286 clean training chunks.
2026-03-09 16:05:23,088 -  > Video eDXGVplLvyg: Generated 374 clean training chunks.
2026-03-09 16:05:23,363 -  > Video qn6I_O8wkFs: Generated 366 clean training chunks.
2026-03-09 16:05:23,529 -  > Video 2XZ9z6OewR0: Generated 287 clean training chunks.
2026-03-09 16:05:23,681 -  > Video wr8VVJRcRSg: Generated 232 clean training chunks.
2026-03-09 16:05:23,796 -  > Video fdDJub69Hnk: Generated 330 clean training chunks.
2026-03-09 16:05:23,979 -  > Video eVFzbxmKNUw: Generated 275 clean training chunks.
2026-03-09 16:05:24,320 -  > Video oel9-7Az0vw: Generated 267 clean training chunks.
2026-03-09 16:05:24,463 -  > Video d-SWWnl3WLM: Generated 196 clean training chunks.
2026-03-09 16:05:24,560 -  > Video f_N3PGvnVKg: Generated 148 clean training chunks.
2026-03-09 16:05:24,742 -  > Video 1MoYHJaKsZE: Generated 225 clean training chunks.
2026-03-09 16:05:27,336 -  > Video gYpq1_omxOg: Generated 231 clean training chunks.
2026-03-09 16:05:27,753 -  > Video odLgrFHEAmQ: Generated 347 clean training chunks.
2026-03-09 16:05:27,766 -
🔥 Ready to process 3827 Total Training Chunks!
2026-03-09 16:05:27,789 - Extracting audio matrices... (This is CPU bound and may take 20-30+ minutes for huge 
Map: 100% 3827/3827 [00:27<00:00, 138.07 examples/s]
2026-03-09 16:05:55,511 - Tokenizing words to integer matrices...
Map: 100% 3827/3827 [03:38<00:00, 17.55 examples/s]
2026-03-09 16:09:34,138 - Wiping intermediate disk files to free storage space...
Saving the dataset (7/7 shards): 100% 3444/3444 [01:47<00:00, 31.90 examples/s]
Saving the dataset (1/1 shards): 100% 383/383 [00:20<00:00, 18.71 examples/s]
2026-03-09 16:11:42,666 -
✅ Phase 2: Processing Complete! Dataset safely housed mathematically at: /content/dataset/processed/whisper da
```

```
!pip install -U transformers datasets accelerate evaluate
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Collecting transformers
  Downloading transformers-5.3.0-py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Collecting datasets
  Downloading datasets-4.6.1-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.12.0)
Collecting accelerate
  Downloading accelerate-1.13.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: evaluate in /usr/local/lib/python3.12/dist-packages (0.4.6)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.
```

```
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.24.3)
Collecting pyarrow>=21.0.0 (from datasets)
  Downloading pyarrow-23.0.1-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (3.1 kB)
Requirement already satisfied: dill<0.4.1,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.3
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.19 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: fsspec<=2026.2.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fss
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.10.
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fssp
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (4
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->data
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->datasets) (3.
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingfac
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggin
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from reques
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accele
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->acc
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->a
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=2.
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=
```

Step 5: GPU Model Fine-Tuning

Configure your batch size and steps. We use FP16 Mixed Precision for the fastest training speeds to fully utilize the T4 Tensor Cores.

```
%%writefile 3_train_whisper.py

import os
import torch
import logging
from dataclasses import dataclass
from typing import Any

from datasets import load_from_disk
from transformers import (
    WhisperForConditionalGeneration,
    WhisperProcessor,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer
)

import evaluate

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ------------------------------------------------
# Paths
# ------------------------------------------------

BASE_DIR = os.getcwd()

PROCESSED_DATA_DIR = os.path.join(
    BASE_DIR,
    "dataset",
    "processed",
    "whisper_dataset"
)

DRIVE_PATH = "/content/drive/MyDrive/MeetBuddy_Whisper_Models"
```

```
if os.path.exists("/content/drive/MyDrive"):
    os.makedirs(DRIVE_PATH, exist_ok=True)
    OUTPUT_DIR = os.path.join(
        DRIVE_PATH,
        "meetbuddy-whisper-small-finetuned"
    )
else:
    OUTPUT_DIR = os.path.join(
        BASE_DIR,
        "meetbuddy-whisper-small-finetuned"
    )

MODEL_ID = "openai/whisper-small"


# --------------------------------------------------
# Data Collator
# --------------------------------------------------

@dataclass
class DataCollatorSpeechSeq2SeqWithPadding:

    processor: Any

    def __call__(self, features):

        input_features = [
            {"input_features": f["input_features"]}
            for f in features
        ]

        batch = self.processor.feature_extractor.pad(
            input_features,
            return_tensors="pt"
        )

        label_features = [
            {"input_ids": f["labels"]}
            for f in features
        ]

        labels_batch = self.processor.tokenizer.pad(
            label_features,
            return_tensors="pt"
        )

        labels = labels_batch["input_ids"].masked_fill(
            labels_batch.attention_mask.ne(1),
            -100
        )

        if (
            labels[:, 0]
            == self.processor.tokenizer.bos_token_id
        ).all().cpu().item():
            labels = labels[:, 1:]

        batch["labels"] = labels

        return batch


# --------------------------------------------------
# Training
# --------------------------------------------------

def main():

    logger.info(
        f"Loading dataset from {PROCESSED_DATA_DIR}"
    )

    dataset = load_from_disk(PROCESSED_DATA_DIR)

    processor = WhisperProcessor.from_pretrained(
        MODEL_ID,
        language="English",
        task="transcribe"
    )

    model = WhisperForConditionalGeneration.from_pretrained(
        MODEL_ID
```

```
)

model.config.forced_decoder_ids = None
model.config.suppress_tokens = []

data_collator = DataCollatorSpeechSeq2SeqWithPadding(
    processor=processor
)

metric = evaluate.load("wer")


# ------------------------------------------------
# Metrics
# ------------------------------------------------

def compute_metrics(pred):

    pred_ids = pred.predictions
    label_ids = pred.label_ids

    label_ids[label_ids == -100] = (
        processor.tokenizer.pad_token_id
    )

    pred_str = processor.tokenizer.batch_decode(
        pred_ids,
        skip_special_tokens=True
    )

    label_str = processor.tokenizer.batch_decode(
        label_ids,
        skip_special_tokens=True
    )

    wer = 100 * metric.compute(
        predictions=pred_str,
        references=label_str
    )

    return {"wer": wer}


# ------------------------------------------------
# Training Arguments
# ------------------------------------------------

training_args = Seq2SeqTrainingArguments(

    output_dir=OUTPUT_DIR,

    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,

    gradient_accumulation_steps=2,

    learning_rate=1e-5,

    warmup_steps=100,

    max_steps=3000,

    gradient_checkpointing=True,

    fp16=True,

    eval_strategy="steps",

    predict_with_generate=True,

    generation_max_length=225,

    save_steps=200,

    eval_steps=200,

    logging_steps=50,

    load_best_model_at_end=True,

    metric_for_best_model="wer",
```

```
        greater_is_better=False,

        save_total_limit=2,

        remove_unused_columns=False
    )


    # ------------------------------------------------
    # Trainer
    # ------------------------------------------------

    trainer = Seq2SeqTrainer(

        args=training_args,

        model=model,

        train_dataset=dataset["train"],

        eval_dataset=dataset["test"],

        data_collator=data_collator,

        compute_metrics=compute_metrics,

        processing_class=processor
    )


    torch.set_float32_matmul_precision("high")

    logger.info(
        "\n🔥 STARTING WHISPER TRAINING 🔥"
    )

    trainer.train()


    # ------------------------------------------------
    # Save Model
    # ------------------------------------------------

    model.save_pretrained(OUTPUT_DIR)

    processor.save_pretrained(OUTPUT_DIR)

    logger.info(
        f"\n✅ Model saved to {OUTPUT_DIR}"
    )


# ------------------------------------------------

if __name__ == "__main__":

    main()
```

```
Overwriting 3_train_whisper.py
```

```
!python 3_train_whisper.py
```

```
INFO:__main__:Loading dataset from /content/dataset/processed/whisper_dataset
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/openai/whisper-small/tree/main/additional_chat_tem
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/processor_config.json "HTT
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/chat_template.json "HTTP/1
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/chat_template.jinja "HTTP/
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/audio_tokenizer_config.jsc
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/processor_config.json "HTT
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/preprocessor_config.json "
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/processor_config.json "HTT
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/preprocessor_config.json "
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/config.json "HTTP/1.1 307
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/tokenizer_config.json "HTT
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/openai/whisper-small/tree/main/additional_chat_tem
INFO:httpx:HTTP Request: GET https://huggingface.co/api/models/openai/whisper-small/tree/main?recursive=true&expa
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/config.json "HTTP/1.1 307
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
```

```
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/config.json "HTTP/1.1 307
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/model.safetensors "HTTP/1.
Loading weights: 100% 479/479 [00:00<00:00, 4821.86it/s]
INFO:httpx:HTTP Request: HEAD https://huggingface.co/openai/whisper-small/resolve/main/generation_config.json "HT
INFO:httpx:HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/openai/whisper-small/973afd24965f72
INFO:__main__:
🔥 STARTING WHISPER TRAINING 🔥
  0% 11/3000 [00:51<3:43:57,  4.50s/it]
```

Step 6: Backup Download

If you did NOT connect Google Drive in Step 0, run this cell to download the model as a Zip file directly to your computer.

```
!zip -r meetbuddy-whisper-small-finetuned.zip meetbuddy-whisper-small-finetuned/
from google.colab import files
files.download('meetbuddy-whisper-small-finetuned.zip')
```